서술형 예상 16문제 + 모범답안 핵심

실제 시험: 1~3번 파일 + 04번 A~E에서 9문제, F·G에서 1문제 (Q1~10은 1순위, Q11~16은 대비용) · **빨간 글씨** = 답안에 꼭 들어가야 할 키워드

Q1. OpenCV로 읽은 이미지를 Matplotlib으로 출력하면 색이 이상하게 나온다. 이유와 해결 방법을 쓰고, RGB 이미지를 BGR2GRAY로 변환하면 결과가 달라지는 이유도 설명하시오. 2. CV

- OpenCV는 **BGR**, Matplotlib은 **RGB** 순서로 해석 → R과 B가 뒤바뀌어 보임
- 해결: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` 로 시각화 도구의 기준에 맞춰 변환
- **Gray = 0.299R + 0.587G + 0.114B** — 채널마다 가중치가 다름. 채널 순서를 잘못 알면 R과 B의 가중치가 바뀌어 밝기가 달라짐

Q2. HSV와 LAB 색공간을 비교하고, 조명 오염이 심한 이미지에서 LAB + CLAHE를 쓰는 이유를 HE → AHE → CLAHE 발전 과정과 함께 설명하시오. 2. CV

- HSV: 색상·채도·명도가 **독립** → 색 추출이 쉬움. 다만 조명 변화에 취약
- LAB: **대립색 이론** 기반. L(지각 밝기)과 a·b(색)가 분리 → **조명에 강함**
- HE: 전역 재분배 → 일부 영역이 과도하게 밝아지거나 어두워짐
- AHE: 격자(tile)별 처리 → 국소 대비는 향상되지만 단색 영역의 노이즈 증폭
- CLAHE: 히스토그램 상한(**clipLimit**)을 넘는 부분을 잘라 재분배 → 노이즈 증폭 방지
- **L 채널에만** 적용 → 색은 그대로 두고 밝기 대비만 강화

Q3. 이미지의 저주파와 고주파를 설명하고, Sobel Filter의 한계와 이를 보완한 Canny Edge Detection의 4단계를 서술하시오. 2. CV

- 저주파: 밝기가 완만하게 변하는 영역(형태, 배경) / 고주파: 급격하게 변하는 영역(경계, 질감, 노이즈). LPF는 블러, HPF는 엣지를 남김
- Sobel: **1차 미분**. dx는 세로 엣지, dy는 가로 엣지. magnitude = $\sqrt{(dx^2+dy^2)}$. 빠르지만 **노이즈에 약하고 엣지가 두꺼움**
- Canny 4단계:
  1. **Gaussian**으로 노이즈 제거
  2. **Sobel**로 gradient 계산
  3. **NMS**로 국소 최대만 남겨 엣지를 얇게 만듦
  4. **Hysteresis**: 임계값 2개를 쓰고, 중간값은 강한 엣지와 연결된 경우만 남김
- Canny 입력은 uint8만 가능

Q4. 특정 색 객체를 검출하는 과정(Color Segmentation → Morphology → Contour)을 설명하고, Erosion·Dilation·Opening·Closing의 차이를 쓰시오. 2. CV

- 파이프라인: 약한 블러 → HSV 변환 → `inRange` 로 **이진 마스크** 생성 → Opening/Closing → `findContours` → **가장 큰 면적**의 contour 선택 → 최소 외접원 또는 무게중심(Cx = M10/M00, Cy = M01/M00)
- 빨강은 H가 0 근처와 180 근처 양쪽에 걸쳐 있어서 마스크 2개를 OR로 합침
- 침식: 커널 안이 모두 1일 때만 1 (**AND**) → 흰 영역 축소 / 팽창: 하나라도 1이면 1 (**OR**) → 흰 영역 확장
- Opening(침식 → 팽창)은 **배경 노이즈 제거**, Closing(팽창 → 침식)은 **내부 구멍 메움**

Q5. 단층 퍼셉트론과 다층 퍼셉트론의 차이를 XOR 문제를 예로 설명하시오. 3. DL

- 퍼셉트론: z = Σwx + b, y = f(z). 가중치는 중요도, 편향은 활성화 기준
- 단층은 결정 경계가 직선 → **선형 분리** 문제(AND, OR, NAND, NOR)만 해결 가능
- XOR은 **비선형** → 단층으로 불가능. 학습해도 확률이 약 0.5, 정확도 50%에 머묾
- **은닉층**을 추가한 MLP는 비선형 분리 가능. XOR = AND(NAND, OR)
- 모델 구조가 문제에 맞지 않으면 epoch나 데이터를 늘려도 해결되지 않음

Q6. 인공신경망의 학습 과정을 손실 함수, 순전파, 역전파, 경사하강법, 학습률, Epoch 개념을 사용해 설명하시오. 3. DL

- 흐름: 순전파로 예측 → **손실 함수**로 정답과의 오차 계산 → **역전파**로 연쇄법칙(dL/dw = dL/dy·dy/dz·dz/dw)을 이용해 기울기 계산 → **w ← w − η·dL/dw**
- 학습률이 작으면 느리고, 크면 최적값을 지나치거나 발산·불안정
- Epoch가 적으면 **과소적합**, 많으면 **과적합**
- 활성화 함수가 비선형이어야 층을 쌓는 의미가 있음. Step 함수는 미분값이 0이라 학습에 쓸 수 없음

Q7. CNN의 구성 요소(Convolution, Feature Map, ReLU, Pooling, FC)의 역할을 설명하고, 전통적인 필터링(Sobel 등)과의 차이를 쓰시오. 3. DL

- 공간 구조를 유지한 채 커널 합성곱으로 특징 추출 → **Feature Map** (커널 1개당 1장)
- ReLU는 음수 반응 제거 / Pooling은 크기를 줄여 효율과 위치 변화에 대한 강건성 향상 / FC는 최종 분류
- 얕은 층은 선·경계, 깊은 층은 질감·형태를 잡음
- 차이점: 전통 필터는 사람이 커널을 설계하지만, CNN은 커널(가중치)을 데이터로 **학습**
- 출력 크기 **⌊(H+2P−K)/S⌋+1** 계산이 함께 나올 수 있음

Q8. CIFAR-10 학습에서 사용한 데이터 증강과 정규화의 목적, 적용 대상의 차이를 설명하시오. (BatchNorm, Dropout 포함 가능) 3. DL

- 데이터 증강(RandomCrop, Flip): 다양성을 높여 **과적합 완화**. **학습 데이터에만** 적용
- 정규화(채널별 mean·std): 입력 분포를 맞춰 **gradient 안정화**. **학습·테스트 모두**에 같은 기준으로 적용
- BatchNorm: 각 층의 출력 분포를 정규화해 학습 안정화
- Dropout: 학습 중 일부 뉴런을 무작위로 꺼서 특정 특징에 의존하지 않게 함 → 과적합 완화

Q9. IoU의 정의를 쓰고, 객체 탐지에서 NMS의 필요성과 동작 과정을 설명하시오. YOLO가 실시간 처리에 적합한 이유도 쓰시오. 4. OD A~E

- **IoU = 교집합 면적 / 합집합 면적**. 범위 0~1 (0 = 전혀 겹치지 않음, 1 = 완전히 동일)
- NMS가 필요한 이유: 같은 객체에 BBox가 여러 개 겹쳐서 예측되기 때문
- NMS 과정:
  1. Confidence 임계값 미만인 박스 제거
  2. 가장 높은 confidence의 박스 선택
  3. 나머지 박스와 IoU 계산
  4. IoU가 임계값 초과인 박스 제거
  5. 남은 박스로 반복
- YOLO는 **One-stage** 탐지기: 이미지를 한 번만 통과시켜 BBox, Class, Confidence를 동시에 예측. **Region Proposal 단계가 없음**. Feature Map을 공유하고 여러 위치를 병렬로 예측해 GPU를 효율적으로 사용

Q10. 추론 FPS와 End-to-End FPS의 차이, FPS에 EMA를 적용하는 이유를 설명하시오. 또 Jetson의 Power Mode를 무조건 최고로 설정하면 안 되는 이유를 쓰시오. 4. OD F·G

- FPS = 1 / 처리 시간(s)
- 추론 FPS는 모델 forward 속도만 잰 것 / **End-to-End FPS**는 입력·전처리·추론·NMS·시각화·출력을 모두 포함한 체감 속도로, 실제 성능 평가에 사용
- **EMA** (`0.9·이전 + 0.1·현재`): 순간적인 병목이나 프레임 드롭으로 생기는 급격한 변동을 완화하면서 최신 값도 반영
- Power Mode (0: 15W, 1: 25W, 2: MAXN SUPER)를 높이면 성능은 오르지만 전력과 발열이 증가:
  - 열이나 전력 한계에 도달하면 **Throttling**으로 클럭이 제한되어 성능이 오히려 저하
  - 심하면 하드웨어 보호를 위해 강제 **Shutdown**
  - 그래서 필요한 성능, 전원, 배터리, 냉각 환경을 고려해서 선택
- 관련 명령어: `nvpmodel -q` 확인 / `nvpmodel -m N` 변경 / `tegrastats` 모니터링 (RAM, CPU, GR3D_FREQ = GPU, cpu@·gpu@ = 온도, VDD_IN = 전체 전력)

Q11. 규칙 기반 객체 탐지와 학습 기반 객체 탐지를 비교하시오. (객체 탐지의 정의 포함) 4. OD 도입

- 객체 탐지: 이미지나 영상에서 객체의 **위치(BBox)**와 **종류(Class)**를 찾는 기술. 위치 추정(Localization)과 분류(Classification)의 결합이며, 여러 객체를 동시에 인식 가능
- **규칙 기반** (예: Color Segmentation → Contour → `boundingRect`):
  - 사람이 색상·크기·형태 규칙을 직접 설정 → 학습 데이터가 필요 없음
  - 환경이 바뀌면 규칙을 다시 고쳐야 함
  - 조명·배경·크기·방향 변화에 민감
- **학습 기반** (예: YOLO):
  - 모델이 데이터로부터 특징을 학습
  - 많은 데이터와 연산이 필요
  - 복잡한 배경과 다양한 객체 변화에 대응 가능
  - 내부 판단 과정을 해석하기 어려움

Q12. Bounding Box의 4가지 표현 방식을 설명하고, (cx, cy, w, h)를 (x1, y1, x2, y2)로 변환하는 방법을 쓰시오. 4. OD A

- BBox: 탐지된 객체를 둘러싸는 직사각형. OpenCV 좌표계 기준(원점은 좌측 상단)
- ① **(x1, y1, x2, y2)**: 좌상단과 우하단 좌표. 너비 = x2−x1, 높이 = y2−y1. `cv2.rectangle`로 그리기 편리
- ② **(x, y, w, h)**: 좌상단 좌표와 크기. 우하단 = (x+w, y+h). `boundingRect`, `selectROI`, `NMSBoxes`에서 사용
- ③ **(cx, cy, w, h)**: 중심 좌표와 크기
- ④ **정규화**: cx/W, cy/H, w/W, h/H → 0~1 범위. 해상도와 무관하게 표현 가능
- 변환: x1 = cx − w/2, y1 = cy − h/2, x2 = cx + w/2, y2 = cy + h/2
- 예: (100, 80, 40, 20) → (80, 70, 120, 90)

Q13. Kalman Filter의 개념과 동작 과정(예측·보정)을 설명하고, 객체가 가려졌을 때 어떻게 추적을 유지하는지 쓰시오.

- 노이즈가 섞인 측정값으로부터 실제 상태(위치, 속도)를 추정하는 알고리즘. 로봇, 자율주행, 객체 추적 등에 활용
- **예측(Prediction)**: 운동 모델(등속 가정)로 다음 상태와 불확실성을 예측
- **보정(Update)**: 측정값이 들어오면 **칼만 이득** K로 예측과 측정을 가중 결합
- 예측과 보정을 반복하면서 추정값을 계속 개선
- 객체가 **가려지거나 측정에 실패하면 예측만 수행** → 추적이 끊기지 않음
- 노이즈 설정: 프로세스 노이즈가 크면 측정 변화에 민감하게 반응, 측정 노이즈가 크면 측정을 덜 믿고 예측을 더 따름
- OpenCV 예: 상태 [x, y, vx, vy], 측정 [x, y] → `KalmanFilter(4, 2)`

Q14. NumPy(CPU)와 CuPy(GPU)의 차이를 메모리와 연산 방식 측면에서 설명하고, GPU 시간 측정 시 주의할 점과 작은 배열에서 GPU가 더 느린 이유를 쓰시오. 3. DL

- NumPy 배열은 **RAM(host)**, CuPy 배열은 **VRAM(device)**에 저장. 문법은 비슷하지만 연산이 실행되는 장치가 다름
- CPU는 소수의 고성능 코어(Orin Nano 6개)로 순차 처리, GPU는 다수의 코어(1024개)로 대규모 병렬 처리
- Jetson은 **UMA(통합 메모리)**로 물리 칩을 공유하지만 논리 영역은 분리 → `cp.asarray` / `cp.asnumpy`로 **복사 필요**. 둘을 섞어서 연산하면 에러
- 시간 측정 시 주의:
  - GPU는 비동기로 실행되므로 **synchronize**로 작업 완료를 기다린 뒤 측정
  - 첫 실행에는 초기화 비용이 들어서 **warm-up** 필요
- 작은 배열에서는 데이터 전송과 커널 실행의 **오버헤드**가 병렬 연산 이득보다 커서 오히려 느림
- GPU 병렬화 방식: 출력 픽셀이 아니라 커널 원소(이동량)마다 반복하면서 배열 전체를 한 번에 연산

Q15. 일반 USB 카메라와 Jetson CSI 카메라의 차이를 설명하고, CSI 카메라에서 nvarguscamerasrc를 사용하는 이유를 쓰시오. 2. CV

- USB 카메라: 이미 **MJPG**나 **YUYV**로 인코딩된 영상을 출력 → `v4l2src`로 바로 받아서 디코딩·변환
- CSI 카메라: 센서의 원본 **Bayer 데이터**(RG10, 10-bit)를 출력 → **ISP**(Image Signal Processor) 처리가 필요
- **nvarguscamerasrc**: NVIDIA Argus 인터페이스. Jetson ISP로 Bayer 데이터를 처리해서 OpenCV가 쓸 수 있는 BGR 이미지로 변환
- GStreamer 파이프라인: 처리 단계를 `!`로 연결 → `VideoCapture(pipeline, CAP_GSTREAMER)`. `appsink drop=true max-buffers=1`, leaky queue로 오래된 프레임을 버려 지연 감소
- 포맷 확인은 `v4l2-ctl --list-formats-ext`, 오류가 나면 `nvargus-daemon` 재시작

Q16. Canny Edge Detection의 NMS와 객체 탐지(YOLO)의 NMS를 비교하시오. 2. CV + 4. OD

- 공통점: "최대값만 남기고 나머지는 억제"해서 **중복을 제거**한다는 원리
- **Canny NMS**:
  - 대상은 **픽셀**, 기준은 gradient 크기
  - gradient 방향으로 이웃 픽셀과 비교해서 국소 최대가 아니면 0으로 만듦
  - 목적: 두꺼운 엣지를 1픽셀 두께로 얇게 만들기
- **객체 탐지 NMS**:
  - 대상은 **Bounding Box**, 기준은 Confidence와 **IoU**
  - 가장 높은 confidence의 박스를 고르고, 그 박스와 IoU가 임계값을 넘는 박스를 제거하는 과정을 반복
  - 목적: 객체당 대표 박스 1개만 남기기
- 참고: Harris Corner 검출의 마지막 단계에도 NMS가 사용됨